Teoria Współbieżności

Kajetan Frątczak 11.10.2025 r.

Laboratorium 02

Wyścig wątków - semafory

1. Treść zadań

Zadanie 1

1. Zaimplementować semafor binarny za pomocą metod wait i notify, użyć go do synchronizacji programu Wyścig.

Zadanie 2

2. Pokazać, że do implementacji semafora za pomocą metod wait i notify nie wystarczy instrukcja if tylko potrzeba użyć while . Wyjaśnić teoretycznie dlaczego i potwierdzić eksperymentem w praktyce. (wskazówka: rozważyć dwie kolejki: czekająca na wejście do monitora obiektu oraz kolejkę związaną z instrukcja wait , rozważyć kto kiedy jest budzony i kiedy następuje wyścig).

Zadanie 3

3. Zaimplementować semafor licznikowy (ogólny) za pomocą semaforów binarnych. Czy semafor binarny jest szczególnym przypadkiem semafora ogólnego ?

2. Argumentacja rozwiązania

Zadanie 1

Semafor binarny zaimplementowałem w Javie na podstawie zdefiniowanych operacji z konspektu do laboratorium, przy użyciu metod wait() i notify().

Zadanie 2

Semafor zmodyfikowałem tak, aby używał *if* zamiast *while* i sprawdziłem co się stanie. Okazuje się, że wtedy semafor niepoprawnie synchronizuje nasz program. Powodem jest to, że *if* zakłada, że po obudzeniu wątek się już zmienił. W *if* warunek sprawdzany jest tylko raz przed pierwszym wait(), zaś w *while* za każdym razem po obudzeniu z wait(). To chroni przed sytuacją, gdy między obudzeniem a zdobyciem monitora inny wątek zmieni stan.

Zadanie 3

Semafor binarny jest szczególnym przypadkiem semafora ogólnego. Semafor ogólny ma wartość początkową n >= 0 i pozwala n wątkom jednocześnie korzystać z zasobów (pula zasobów), podczas gdy semafor binarny to semafor ogólny z n = 1, który zapewnia wzajemne wykluczanie – tylko jeden wątek może być w sekcji krytycznej.

3. Rozwiązanie

Zadanie 1

```java
// semafor binarny do synchronizacji watkow
class Semafor {
   private boolean _stan = true; // mówi, czy semafor jest wolny
   private int _czeka = 0; // liczba czekających wątków

   public Semafor() {

   }

   // Semafor binarny - w skrócie to zmienna sterująca dostępem do sekcji krytycznej (0 lub 1).
   // wait() - usypia wątek i zwalnia monitor obiektu
   // notify() - budzi wątek
   // P - opuszczenie, V - podniesienie
   // w P zajmujemy sekcję krytyczną, a w V zwalniamy
   // P: jeśli S == 1, ustaw na 0 i kontynuuj, jeśli S==0, zablokuj wątek (czekaj)
   // V: jeśli jakieś wątki oczekują (1 lub 2) - obudź jeden z nich, w przeciwnym razie ustaw S=1

   public synchronized void P() {
       while (!_stan) {
           _czeka++;
           try {
               wait();
           } catch (InterruptedException e) {}
           _czeka--;
       }
       _stan = false;
   }

   public synchronized void V() {
       if (_czeka > 0) {
           notify();
       }
       _stan = true;
   }
}

class Counter {
   private int _val;
   Semafor s = new Semafor();

   public Counter(int n) {
       _val = n;
   }
   public void inc() {
       s.P();
       _val++;
       s.V();
   }
   public void dec() {
       s.P();
       _val--;
       s.V();

   }
   public int value() {
       return _val;
   }
}
```

Zadanie 2

```java
// semafor binarny do synchronizacji watkow
class Semafor {
   private boolean _stan = true; // mówi, czy semafor jest wolny
   private int _czeka = 0; // liczba czekających wątków

   public Semafor() {

   }

   // Semafor binarny - w skrócie to zmienna sterująca dostępem do sekcji krytycznej (0 lub 1).
   // wait() - usypia wątek i zwalnia monitor obiektu
   // notify() - budzi wątek
   // P - opuszczenie, V - podniesienie
   // w P zajmujemy sekcję krytyczną, a w V zwalniamy
   // P: jeśli S == 1, ustaw na 0 i kontynuuj, jeśli S==0, zablokuj wątek (czekaj)
   // V: jeśli jakieś wątki oczekują (1 lub 2) - obudź jeden z nich, w przeciwnym razie ustaw S=1

   public synchronized void P() {
       if (!_stan) {
           _czeka++;
           try {
               wait();
           } catch (InterruptedException e) {}
           _czeka--;
       }
       _stan = false;
   }

   public synchronized void V() {
       if (_czeka > 0) {
           notify();
       }
       _stan = true;
   }
}
class Counter {
   private int _val;
   Semafor s = new Semafor();

   public Counter(int n) {
       _val = n;
   }
   public void inc() {
       s.P();
       _val++;
       s.V();
   }
   public void dec() {
       s.P();
       _val--;
       s.V();

   }
   public int value() {
       return _val;
   }
}
```

Zadanie 3

```java
// semafor binarny do synchronizacji watkow
class Semafor {
   private boolean _stan = true; // mówi, czy semafor jest wolny
   private int _czeka = 0; // liczba czekających wątków

   public Semafor() {

   }

   // Semafor binarny - w skrócie to zmienna sterująca dostępem do sekcji krytycznej (0 lub 1).
   // wait() - usypia wątek i zwalnia monitor obiektu
   // notify() - budzi wątek
   // P - opuszczenie, V - podniesienie
   // w P zajmujemy sekcję krytyczną, a w V zwalniamy
   // P: jeśli S == 1, ustaw na 0 i kontynuuj, jeśli S==0, zablokuj wątek (czekaj)
   // V: jeśli jakieś wątki oczekują (1 lub 2) - obudź jeden z nich, w przeciwnym razie ustaw S=1

   public synchronized void P() {
       while (!_stan) {
           _czeka++;
           try {
               wait();
           } catch (InterruptedException e) {}
           _czeka--;
       }
       _stan = false;
   }

   public synchronized void V() {
       if (_czeka > 0) {
           notify();
       }
       _stan = true;
   }
}

// init - ilość dostępnego zasobu - tutaj ilość możliwych wątków
// V - inkrementujemy licznik i jeśli ktoś czeka to budzimy
// P - dekrementujemy licznik i wpuszczamy, a jeśli licznik == 0 wątek musi czekać

class SemaforZliczajacy {
   private int _dost_zasoby = 0; // licznik dostępnych zasobów
   private Semafor mutex; // mutex do ochrony zmiennej _dost_zasoby

   public SemaforZliczajacy(int init) {
       _dost_zasoby = init;
       mutex = new Semafor();
   }

   public void P() {
       mutex.P();
       while (_dost_zasoby == 0) {
           mutex.V();
           try {
               synchronized (this) {
                   wait();
               }
           } catch (InterruptedException e) {}
           mutex.P();
       }
       _dost_zasoby--;
       mutex.V();
   }
```

```java
    public void V() {
        mutex.P();
        _dost_zasoby++;
        if (_dost_zasoby > 0){
            synchronized (this) {
                notify();
            }
        }
        mutex.V();
    }
}

class Counter {
    private int _val;
    SemaforZliczajacy s = new SemaforZliczajacy(1);

    public Counter(int n) {
        _val = n;
    }
    public void inc() {
        s.P();
        _val++;
        s.V();
    }
    public void dec() {
        s.P();
        _val--;
        s.V();

    }
    public int value() {
        return _val;
    }
}
```

4. Wykresy, tabele, wyniki

Zadanie 1

```
Starting Gradle Daemon...
Gradle Daemon started in 2 s 483 ms
> Task :compileJava UP-TO-DATE
> Task :processResources NO-SOURCE
> Task :classes UP-TO-DATE

> Task :Race2.main()
value=0
```

Rysunek 1: Wynik po uruchomieniu programu

Zadanie 2

```
> Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes

> Task :Race2.main()
value=2106
```

Rysunek 2: Wynik po uruchomieniu programu

```
> Task :compileJava UP-TO-DATE
> Task :processResources NO-SOURCE
> Task :classes UP-TO-DATE

> Task :Race2.main()
value=-8353
```

Rysunek 3: Wynik po uruchomieniu programu

- Wątki : T1, T2, T3

Wątek T1 wywołuje P(), _stan = true -> ustawia _stan = false i wchodzi do sekcji krytycznej

Wątek T2 wywołuje P(), _stan = false -> czeka w wait() i _czeka++

Wątek T1 kończy sekcję krytyczną i woła V() budząc wątek T2

Wątek T2 się budzi, ale w tym samym czasie wątek T3 też próbuje wejść do P()

- jeśli T2 się przebudził to jeszcze nie ustawił _stan = false
- T3 sprawdza if(!_stan) -> może być true (bo T1 w V() ustawił na wolny)
- T3 wejdzie i ustawi _stan = false
- T2 też kontynuuje po przebudzeniu, ale nie sprawdza już warunku, bo miał if a nie while

  on też ustawia _stan = false (w przypadku while, nie wszedłby, bo wątek T3 już blokuje)

Dwa wątki wchodzą do sekcji krytycznej -> BŁĄD

Rysunek 4: Wyjaśnienie różnicy między *if* oraz *while*

Zadanie 3

• Semafor binarny

↳ Najprostszy mechanizm synchronizacji - działa jak przełącznik (0/1):

1 (true) -> zasób wolny - można wejść do sekcji krytycznej

0 (false) -> zasób zajęty - trzeba czekać

Jest to implementacja mutexa (mutual exclusion) - tylko jeden wątek na raz może być w sekcji krytycznej.

• Semafor licznikowy (ogólny)

↳ Mechanizm synchronizacji, który kontroluje dostęp do puli zasobów. W przeciwieństwie do semafora binarnego, semafor licznikowy może mieć wartość od 0 do N, gdzie N to liczba dostępnych zasobów.

• Porównanie

```
class BinarySemafor {
    private boolean _stan = true;
    private int _czeka = 0;
}
```

```
class CountingSemafor {
    private int _dost_zasoby = 0;
    private BinarySemafor mutex;
}
```

Cecha	Binarny	Licznikowy
↳ Wartość	0 lub 1 (boolean)	0 do N (int)
↳ Zasoby	1 zasób	N zasobów
↳ Zastosowanie	Mutex (sekcja krytyczna)	Pula zasobów
↳ Implementacja	Prosta (boolean + wait / notify)	Złożona
↳ Kolejka	1 kolejka oczekujących	1 kolejka oczekujących

↳ P()

1) Sprawdzamy czy semafor jest zajęty
2) Jeśli tak, zwiększamy - czeka
3) Usypiamy wątek (wait())
4) Po wybudzeniu zmniejszamy - czeka
3) Zajmujemy semafor jeśli nie był zajęty

↳ V()

1) Sprawdzamy czy ktoś czeka
2) Jeśli tak, budzimy jeden czekający wątek
3) Zwalniamy semafor

↳ P()

1) Zajmujemy mutex (wchodzimy do sekcji krytycznej)
2) Jeśli brak dostępnych zasobów
   3) Zwalniamy mutex przed czekaniem
   4) Usypiamy wątek
   5) Po obudzeniu znów zajmujemy mutex
3) Zmniejszamy liczbę dostępnych zasobów
4) Zwalniamy mutex

↳ V()

1) Zajmujemy mutex (wchodzimy do sekcji krytycznej)
2) Zwiększamy liczbę dostępnych zasobów
3) Jeśli są zasoby:
   4) Budzimy jeden czekający wątek
4) Zwalniamy mutex

Analogia: P() -> „chcę wjechać na parking" V() -> „wyjeżdżam z parkingu"

Rysunek 5: Wyjaśnienie różnicy oraz sposobu działania semaforów binarnego i liczącego (ogólnego)

```
> Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
> Task :Race2.main()
value=0
```

Rysunek 6: Wynik po uruchomieniu programu

5. Wnioski

- Semafor binarny można skutecznie zaimplementować korzystając z metod wait() i notify(). Dzięki temu możemy synchronizować dostęp do sekcji krytycznej w programie i zapobiegać zjawisku “race condition” - wyścigu wątków. Minusem semafora binarnego jest fakt, iż liczba dostępnych zasobów jest równa 1.
- W semaforze binarnym musimy używać *while* a nie *if* ze względu na to, że wątek po przebudzeniu z wait() nie ma gwarancji, iż warunek nadal jest spełniony - mógł zostać spełniony przez inny wątek, który wcześniej przejął monitor.
- Zadanie 3 udowodniło, że semafor binarny jest szczególnym przypadkiem semafora liczącego (ogólnego), gdzie liczba dostępnych zasobów jest równa 1. Semafor ogólny jest o tyle lepszy, iż pozwala na dostęp do puli zasobów. wielu wątkom na raz
- Metoda synchronizacji wątków za pomocą semaforów jest nieodzowną częścią programowania współbieżnego, choć należy pamiętać o ich odpowiedniej implementacji, aby nie wpaść w pułapkę korzystania z zasobu przez wiele wątków na raz.

6. Bibliografia

https://pl.wikipedia.org/wiki/Semafor_(informatyka)
https://home.agh.edu.pl/~funika/tw/lab2/